JO

Jordanian Legal AI Assistant

Software Requirements Specification

Version 2.0 | IEEE Std 830-1998 (adapted) | 05 April 2026

Document ID	SRS-JLAA-002
Version	2.0 — Post Full Upgrade
Status	Final
Date	05 April 2026
Author	Faris Amjad Lahham
Repository	Faris-amjad-lahham/jordanian-legal-ai-assistant
Standard	IEEE Std 830-1998 (adapted)

Software Requirements Specification (SRS)

Jordanian Legal AI Assistant

Version 2.0 — Post-Upgrade

Field	Value
Document ID	SRS-JLAA-002
Version	2.0
Status	Final
Date	05 April 2026
Author	Faris Amjad Lahham
Repository	`Faris-amjad-lahham/jordanian-legal-ai-assistant`
Standard	IEEE Std 830-1998 (adapted)

Table of Contents

1. Introduction
1. Overall Description
1. Stakeholders & User Classes
1. Functional Requirements
1. Non-Functional Requirements
1. System Architecture
1. Data Requirements
1. External Interface Requirements
1. Diagrams
1. Constraints & Assumptions
1. Appendix

1. Introduction

1.1 Purpose

This document specifies the software requirements for the **Jordanian Legal AI Assistant** (JLAA), a bilingual (Arabic/English) AI-powered web application designed to provide accessible, jurisdiction-aware legal guidance based on **Jordanian Labour Law No. 8 of 1996** and its amendments.

1.2 Scope

The system provides two primary capabilities:

1. **Legal Q&A** — Conversational retrieval-augmented generation (RAG) answering questions about workers' rights, employer obligations, and Jordanian labour regulations.
1. **Contract Intelligence** — Automated generation, editing, and compliance analysis of employment contracts against Jordanian Labour Law.

The system is intended for deployment as a public web application accessible via any modern browser, hosted on Streamlit Community Cloud.

1.3 Definitions & Acronyms

Term	Definition
JLAA	Jordanian Legal AI Assistant
RAG	Retrieval-Augmented Generation — combining document search with LLM generation
BM25	Best Match 25 — lexical text ranking algorithm
LLM	Large Language Model (GPT-4o-mini in this system)
RTL	Right-to-Left — text direction required for Arabic
SRS	Software Requirements Specification
JOD	Jordanian Dinar
SSC	Social Security Corporation (Jordan)
DOCX	Microsoft Word Open XML document format
Embedding	High-dimensional numerical representation of text for semantic similarity
Hybrid Retrieval	Combining lexical (BM25) and semantic (embedding) search scores

1.4 References

- Jordanian Labour Law No. 8 of 1996 and its amendments
- OpenAI API Documentation — GPT-4o-mini, text-embedding-3-small
- Streamlit Documentation v1.32+
- IEEE Std 830-1998: Recommended Practice for SRS
- Social Security Law No. 1 of 2014 (Jordan)

1.5 Overview

This SRS is structured as follows: Section 2 describes the product at a high level. Sections 3–5 detail stakeholders and requirements. Sections 6–9 cover the technical architecture and diagrams. Sections 10–11 list constraints and appendices.

---

2. Overall Description

2.1 Product Perspective

JLAA is a standalone web application that sits at the intersection of:

- **Legal technology** (LegalTech)
- **Conversational AI** (RAG pipeline)
- **Document automation** (contract generation and auditing)

It operates as a **client-server system** where the Streamlit frontend serves as both UI and orchestration layer, delegating to an OpenAI API backend for language model inference and to a local JSON knowledge base for law article retrieval.

```
┌──────────────┐      HTTPS      ┌──────────────────┐     API     ┌──────────────┐
│  User Browser │ ◄──────────► │  Streamlit App   │ ◄──────────► │  OpenAI API
│
└──────────────┘                │  (Cloud/Local)   │             │  GPT-4o-mi
ni │
                                 │                  │             │  Embeddi
ngs  │
                                 │  ┌────────────┐  │             └──────────
──┘
                                 │  │ JSON Data  │  │
                                 │  │ (Law Arts) │  │
                                 │  └────────────┘  │
                                 └──────────────────┘
```

2.2 Product Functions (Summary)

#	Function	Description
F1	Legal Q&A	Bilingual conversational Q&A with sources
F2	Hybrid Retrieval	BM25 + semantic search over law articles
F3	Citation Deep-Dive	View full article text (EN+AR) for any cited article
F4	Know Your Rights	Instant legal fact cards (zero API cost)
F5	Contract Generation	AI-drafted employment contracts
F6	Contract Validation	Real-time rule-based compliance checking
F7	Contract Chat-Edit	Modify contracts via natural language
F8	Contract Upload	Analyze existing PDF/DOCX contracts
F9	Field Extraction	GPT-based structured field extraction from contracts
F10	Load Into Editor	Transfer extracted fields to contract form
F11	PDF Export	Download Q&A session as PDF
F12	Word Export	Download generated contract as DOCX
F13	Rate Limiting	Session-based token budget enforcement
F14	Version History	Restore prior contract versions

2.3 Operating Environment

- **Deployment Platform**: Streamlit Community Cloud (or local Python 3.10+ environment)
- **OS**: Platform-independent (Linux on cloud, Windows/Mac locally)
- **Browser**: Any modern browser (Chrome, Firefox, Safari, Edge)
- **Internet**: Required (OpenAI API calls)
- **Python**: ≥ 3.10

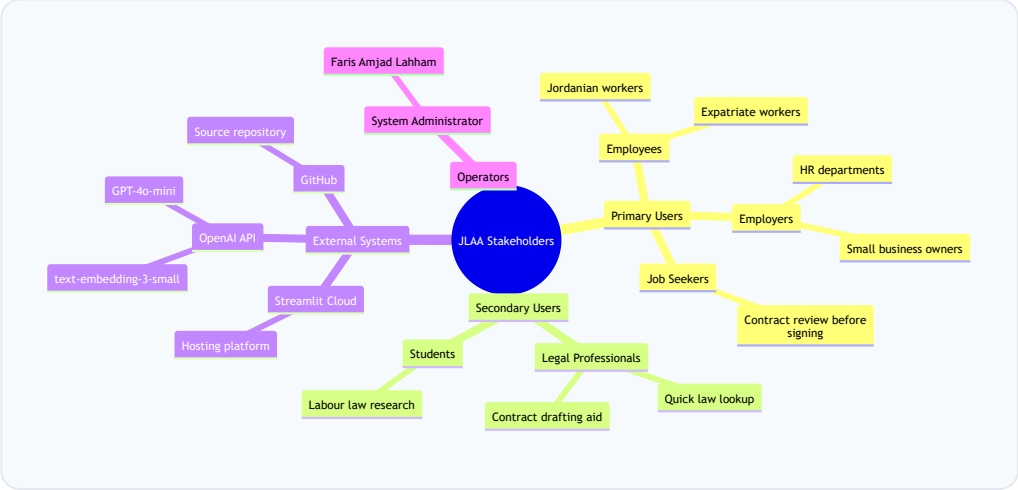
2.4 Constraints

- User data is not persisted between browser sessions (stateless by design)
- Responses are bounded by OpenAI rate limits and quotas
- The system covers **Jordanian Labour Law No. 8/1996 only** — not criminal, civil, or commercial law
- Generated contracts are explicitly **non-binding drafts** for informational purposes only

---

3. Stakeholders & User Classes

3.1 Stakeholder Map



3.2 User Classes

Class	Description	Technical Level	Primary Use
Worker	Employee seeking rights information	Low	Q&A mode
Employer / HR	Drafting or auditing employment contracts	Medium	Contract mode
Legal Professional	Cross-referencing law articles	High	Both modes
Job Seeker	Reviewing a contract before signing	Low	Contract upload + Q&A
Student / Researcher	Learning Jordanian labour law	Low–Medium	Q&A mode

4. Functional Requirements

4.1 Legal Q&A Module

FR-QA-01: Bilingual Query Processing

- The system **shall** accept queries in both Arabic and English.
- Language detection **shall** use the `langdetect` library with a fallback to English.
- The system **shall** respond in the same language as the query unless overridden by the user.

FR-QA-02: Hybrid Retrieval

- Retrieval **shall** combine BM25 lexical scoring (weight: 0.4) and OpenAI embedding cosine similarity (weight: 0.6).
- Article embeddings **shall** be computed on first use and cached to `data/embeddings_{lang}.npy`.
- The system **shall** fall back to BM25-only if the OpenAI embeddings API is unavailable.

FR-QA-03: Confidence Scoring

- Retrieved results **shall** be assigned a hybrid confidence score in [0, 1].
- Scores below 0.5 **shall** trigger a low-confidence warning: *"Verify this information with a licensed Jordanian lawyer."*

FR-QA-04: Streaming Response

- The system **shall** stream GPT responses token-by-token to the UI.
- Arabic responses **shall** be rendered right-to-left (RTL).

FR-QA-05: Citation Deep-Dive

- Every AI response citing articles **shall** display the cited article IDs.
- The user **shall** be able to expand any cited article to view its full text in both English and Arabic side-by-side.

FR-QA-06: Follow-up Suggestions

- After each response, the system **shall** suggest 2–3 relevant follow-up questions.

FR-QA-07: Chat History

- The system **shall** maintain up to 20 messages of conversation context per session.
- The user **shall** be able to clear conversation history.

FR-QA-08: Onboarding Empty State

- When no messages exist, the system **shall** display a welcome panel with 3 English and 3 Arabic sample question chips.
- Clicking a chip **shall** pre-populate and submit it as a query.

FR-QA-09: PDF Export

- The user **shall** be able to download the full Q&A session as a formatted PDF document.
- The PDF **shall** include: timestamp, chat messages with roles, cited articles, confidence flags, and a legal disclaimer.

4.2 Know Your Rights Module

FR-KYR-01: Static Rights Cards

- The system **shall** display a sidebar panel with 10 pre-defined legal facts about Jordanian Labour Law.
- These facts **shall** be hardcoded constants (no API call) for instant, always-correct display.

FR-KYR-02: Accuracy Guarantee

- The displayed values **shall** match the confirmed 2023 Jordanian Labour Law constants.

4.3 Contract Generation Module

FR-CG-01: Contract Form

- The system **shall** provide a structured form with fields: employer, employee, job title, location, contract type (Limited/Unlimited), start date, end date, salary, hours/day, days/week, probation, termination, benefits.

FR-CG-02: Real-time Validation

- The system **shall** validate all contract fields against Jordanian Labour Law constraints in real-time as the user types.
- Validation **shall** produce three severity levels: **error** (blocks generation), **warning** (non-blocking), **info** (advisory).

FR-CG-03: Validation Rules

The following rules **shall** be enforced:

Rule	Severity	Law Reference
Salary < 260 JOD/month	Error	Article 52
Daily hours > 8	Error	Article 56
Weekly hours > 48	Error	Article 56
Probation > 90 days	Error	Article 25
Notice period < 30 days	Warning	Article 32
Ramadan hours > 6/day	Warning	Article 59
Annual leave < 14 days	Warning	Article 61
Missing SSC registration	Info	SSC Law 1/2014
Annual leave progression	Info	Article 61

FR-CG-04: AI Contract Generation

- When no validation errors exist, the system **shall** generate a complete employment contract via GPT-4o-mini.
- The contract **shall** reference relevant Jordanian Labour Law articles.
- The contract **shall** include a legal disclaimer stating it is non-binding.

FR-CG-05: Contract Export

- The user **shall** be able to export the generated contract as a `.docx` Word document.
- The Word document **shall** include proper formatting (headers, fonts, numbered sections).

4.4 Contract Chat-Edit Module

FR-CE-01: Natural Language Editing

- The system **shall** allow the user to modify a generated contract via free-text chat instructions.
- Example: *"Change salary to 800 JOD"*, *"Add a non-compete clause"*, *"Translate to Arabic"*.

FR-CE-02: Version History

- Every edit **shall** save the prior version automatically.
- The user **shall** be able to restore any of the last 10 versions from the sidebar.

4.5 Contract Upload & Analysis Module

FR-CA-01: File Upload

- The system **shall** accept PDF and DOCX files up to 200 MB.
- PDF text extraction **shall** use `pdfplumber`.
- DOCX text extraction **shall** use `python-docx`.

FR-CA-02: Field Extraction

- The system **shall** extract structured contract fields from the raw text using GPT-4o-mini (JSON mode).
- A regex-based fallback **shall** be used when the API is unavailable.

FR-CA-03: Compliance Analysis

- The system **shall** run extracted fields through the same validation engine as the contract form.
- The system **shall** generate an AI narrative summary of compliance status and key risks.

FR-CA-04: Results Display

- The system **shall** display: extracted field table, errors (red), warnings (yellow), info (teal), and AI analysis summary.

FR-CA-05: Load Into Editor

- The user **shall** be able to load extracted fields directly into the Create Contract form with a single click.
- All matching form fields **shall** be pre-populated.

4.6 Rate Limiting Module

FR-RL-01: Session Token Budget

- Each browser session **shall** be allocated:
- Q&A mode: **30,000 tokens**
- Contract mode: **20,000 tokens**

FR-RL-02: Budget Enforcement

- The system **shall** block new API calls when the session budget is exhausted.
- An error message **shall** inform the user to refresh the page.

FR-RL-03: Budget Meter

- The sidebar **shall** display a real-time progress bar showing session token usage.

---

5. Non-Functional Requirements

5.1 Performance

ID	Requirement	Target
NFR-P-01	First Q&A response (first query, no embedding cache)	≤ 15 seconds
NFR-P-02	Q&A response (cached embeddings)	≤ 5 seconds (streaming starts)
NFR-P-03	Contract generation	≤ 10 seconds
NFR-P-04	Contract validation (real-time)	≤ 100 ms (client-side evaluation)
NFR-P-05	Document upload + analysis	≤ 30 seconds
NFR-P-06	App initial load time	≤ 5 seconds

5.2 Reliability

- **NFR-R-01**: The system shall degrade gracefully when the OpenAI API is unavailable — showing an appropriate error and disabling AI-dependent features.
- **NFR-R-02**: BM25 retrieval shall function without any API connection.
- **NFR-R-03**: Contract validation shall function without any API connection.
- **NFR-R-04**: "Know Your Rights" cards shall always display regardless of API availability.

5.3 Security

- **NFR-S-01**: The OpenAI API key shall never be exposed in source code or committed to version control.
- **NFR-S-02**: API keys shall be loaded exclusively from `st.secrets` (cloud) or environment variables (local).
- **NFR-S-03**: Uploaded documents shall not be stored on disk; they shall be processed in-memory only.
- **NFR-S-04**: No user data, queries, or conversations shall be persisted between sessions.

5.4 Usability

- **NFR-U-01**: All UI text shall be available in English; key labels shall support Arabic.
- **NFR-U-02**: Arabic responses shall be automatically rendered in RTL layout.
- **NFR-U-03**: The system shall provide an onboarding experience for first-time users with zero prior knowledge of Jordanian law.
- **NFR-U-04**: Legal citations shall always link to a readable full-text view of the law article.

5.5 Maintainability

- **NFR-M-01**: Law articles shall be stored as structured JSON, updatable without code changes.
- **NFR-M-02**: Validation constants (minimum wage, max hours, etc.) shall be defined as named constants in `contract_backend.py`.
- **NFR-M-03**: Each module (retrieval, contract, utils) shall be independently importable and testable.

5.6 Legal & Ethical

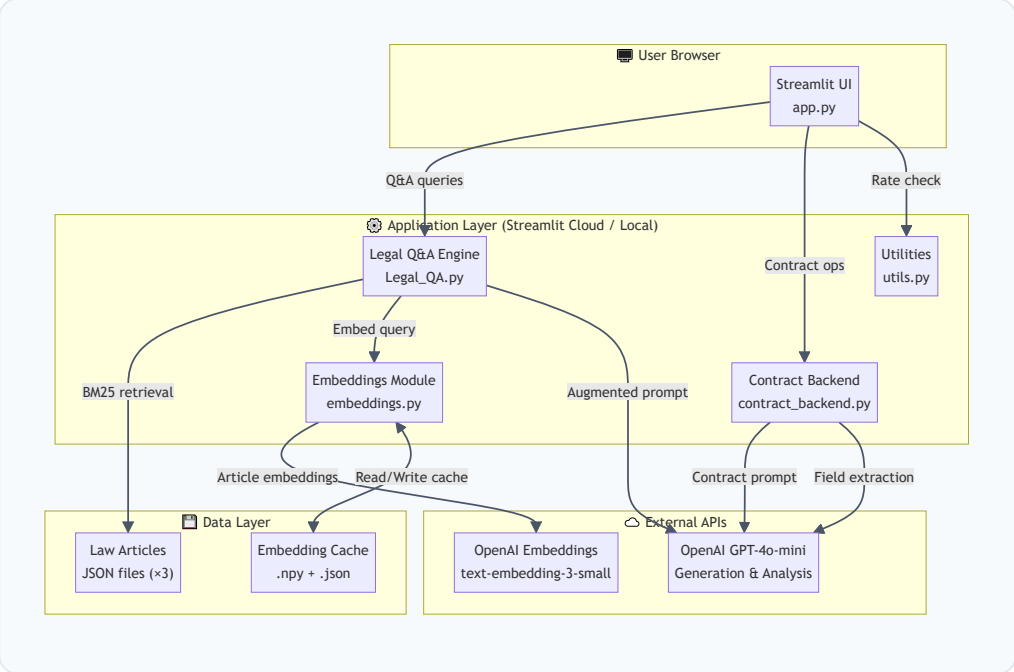
- **NFR-L-01**: Every generated contract shall include a non-binding disclaimer.
- **NFR-L-02**: Every Q&A response with confidence < 0.5 shall include a disclaimer recommending professional legal advice.

- **NFR-L-03**: The system shall not claim to provide legally binding advice at any point.

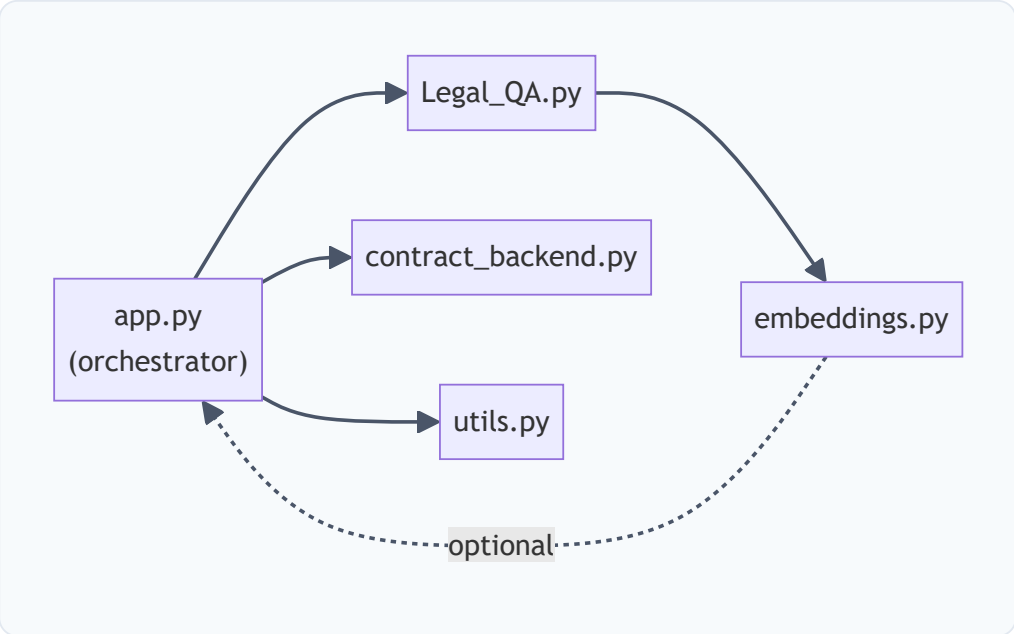
---

6. System Architecture

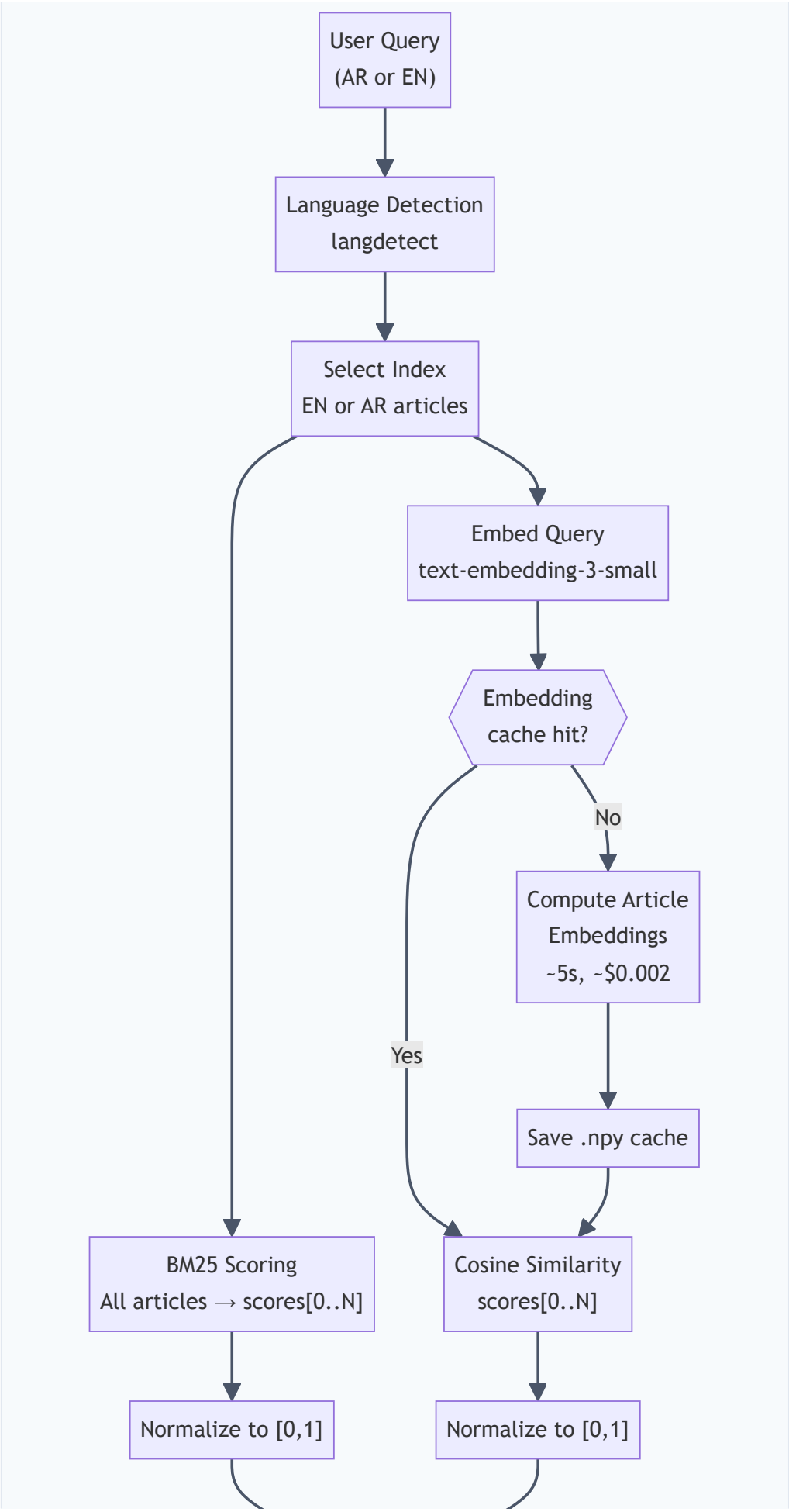
6.1 High-Level Architecture

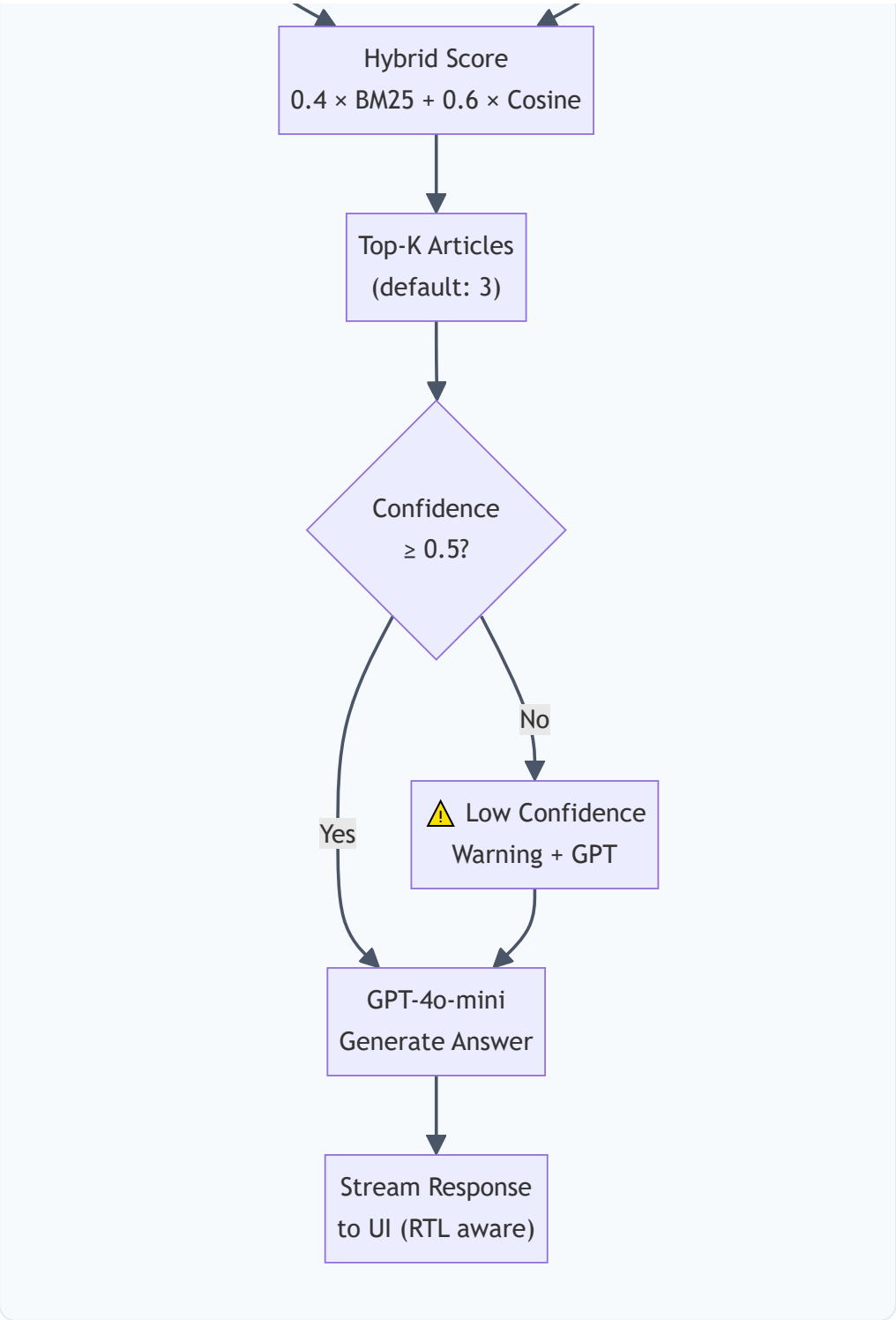


6.2 Module Dependency Map



6.3 Retrieval Pipeline Architecture


User Query
(AR or EN)
Language Detection
langdetect
Select Index
EN or AR articles
Embed Query
text-embedding-3-small
Embedding
cache hit?
No
Compute Article
Embeddings
~5s, ~$0.002
Yes
Save .npy cache
BM25 Scoring
All articles → scores[0..N]
Cosine Similarity
scores[0..N]
Normalize to [0,1]
Normalize to [0,1]

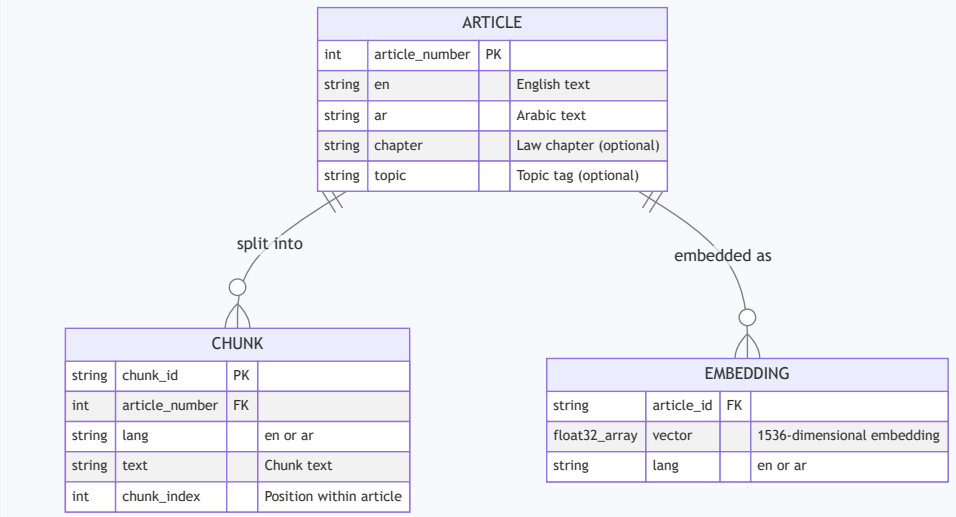


7. Data Requirements

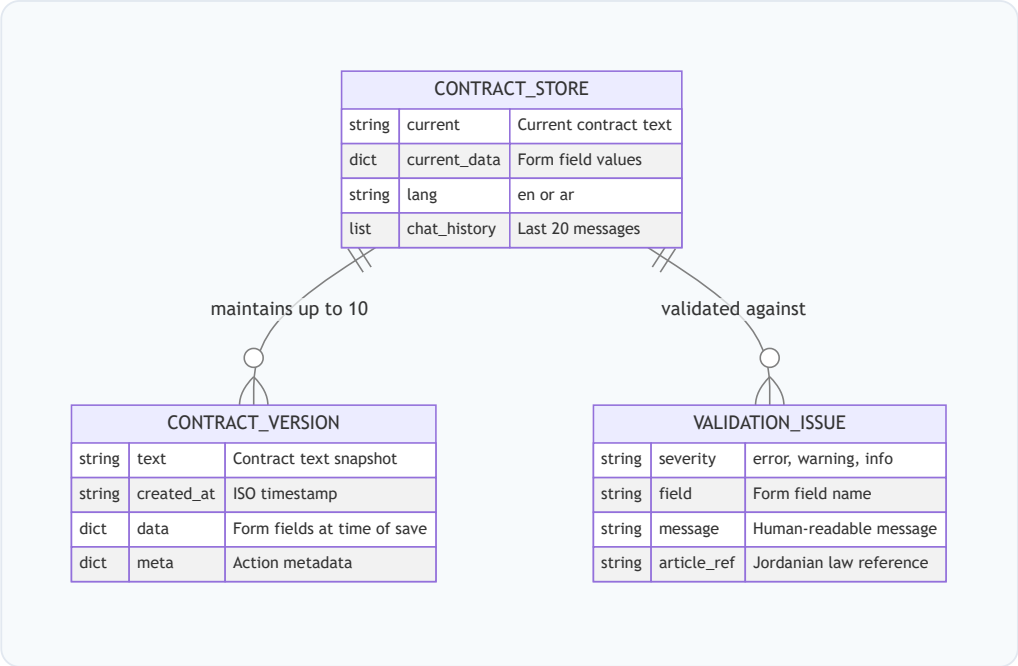
7.1 Data Sources

Source	File	Format	Size	Description
Primary law corpus	`labour_law_articles_multilingual_cleaned.json`	JSON	255 KB	~160 articles, bilingual EN+AR
Chunked corpus	`labour_law_chunks_multilingual_cleaned.json`	JSON	284 KB	Same articles split into chunks for better retrieval
Arabic-only corpus	`labour_law_articles_ar_cleaned.json`	JSON	144 KB	Arabic-only normalized articles
EN embedding cache	`data/embeddings_en.npy`	NumPy binary	1.3 MB	1536-dim vectors, auto-generated
EN embedding index	`data/embeddings_en_ids.json`	JSON	~1 KB	Article ID mapping for cache

7.2 Article Data Schema



7.3 Contract Store Schema (In-Memory)



---

8. External Interface Requirements

8.1 User Interface

- **Framework**: Streamlit 1.32+
- **Layout**: Wide-mode, 2-column (sidebar + main)
- **Theme**: Dark (`#0f1117` background, `#4eb4a1` accent)
- **Typography**: Inter (Google Fonts), fallback sans-serif
- **RTL Support**: Dynamic CSS injection for Arabic responses
- **Responsiveness**: Adapts to desktop and tablet viewports

8.2 OpenAI API Interface

Endpoint	Model	Purpose	Avg Tokens
`chat.completions.create`	`gpt-4o-mini`	Q&A generation	~800 out
`chat.completions.create`	`gpt-4o-mini`	Contract generation	~2000 out
`chat.completions.create`	`gpt-4o-mini`	Field extraction (JSON mode)	~300 out
`chat.completions.create`	`gpt-4o-mini`	Analysis narrative	~200 out
`embeddings.create`	`text-embedding-3-small`	Article embedding	1536 dims
`embeddings.create`	`text-embedding-3-small`	Query embedding	1536 dims

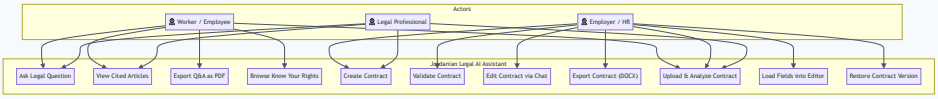
Authentication: Bearer token via `OPENAI_API_KEY` environment variable / `st.secrets`

8.3 File System Interface

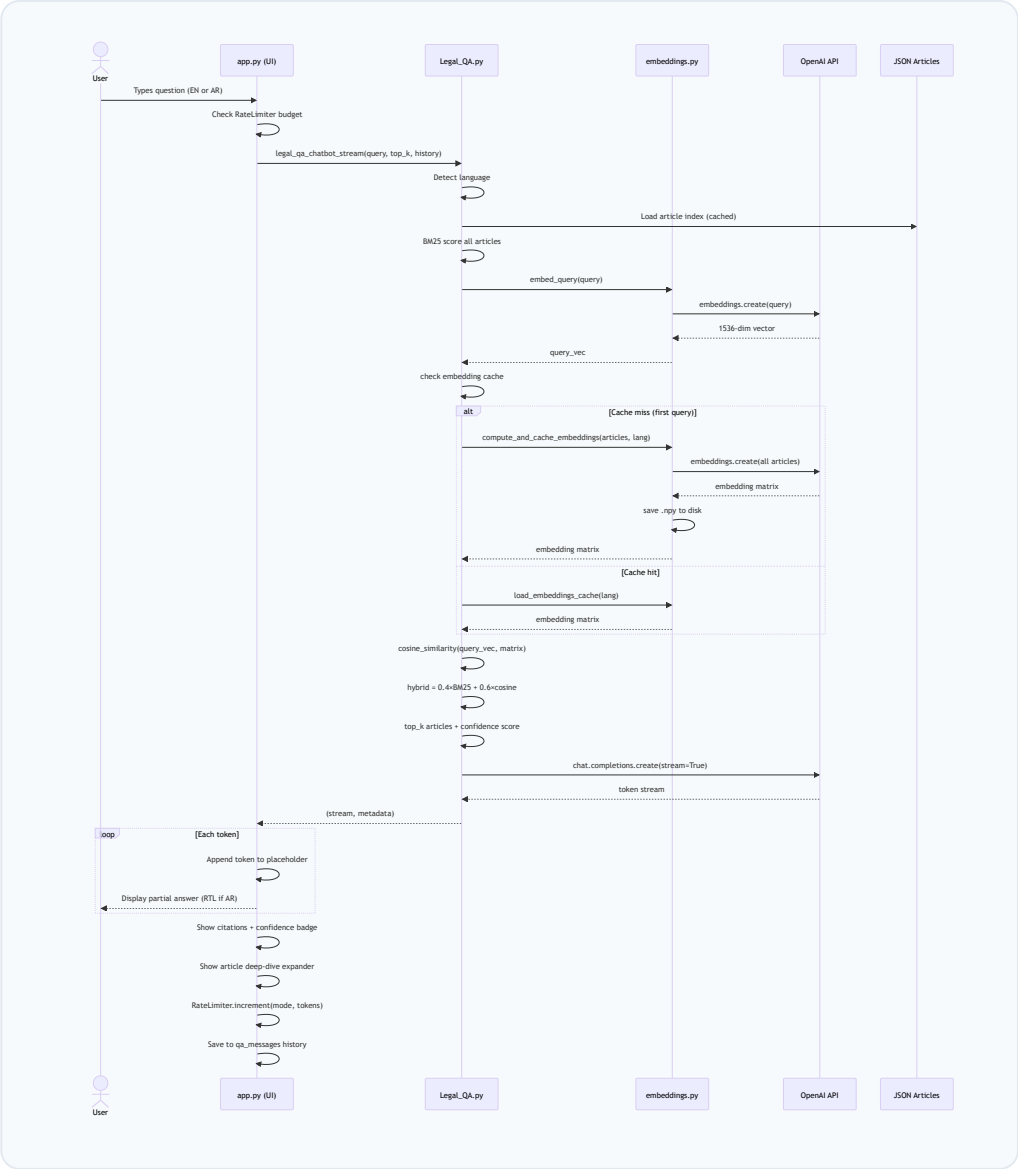
Path	Read/Write	Description
`data/*.json`	Read	Law article datasets
`data/embeddings_{lang}.npy`	Read/Write	Embedding cache
`data/embeddings_{lang}_ids.json`	Read/Write	Embedding ID mapping
`exports/*.docx`	Write	Generated Word contracts
`.streamlit/secrets.toml`	Read	API key (local dev only)

9. Diagrams

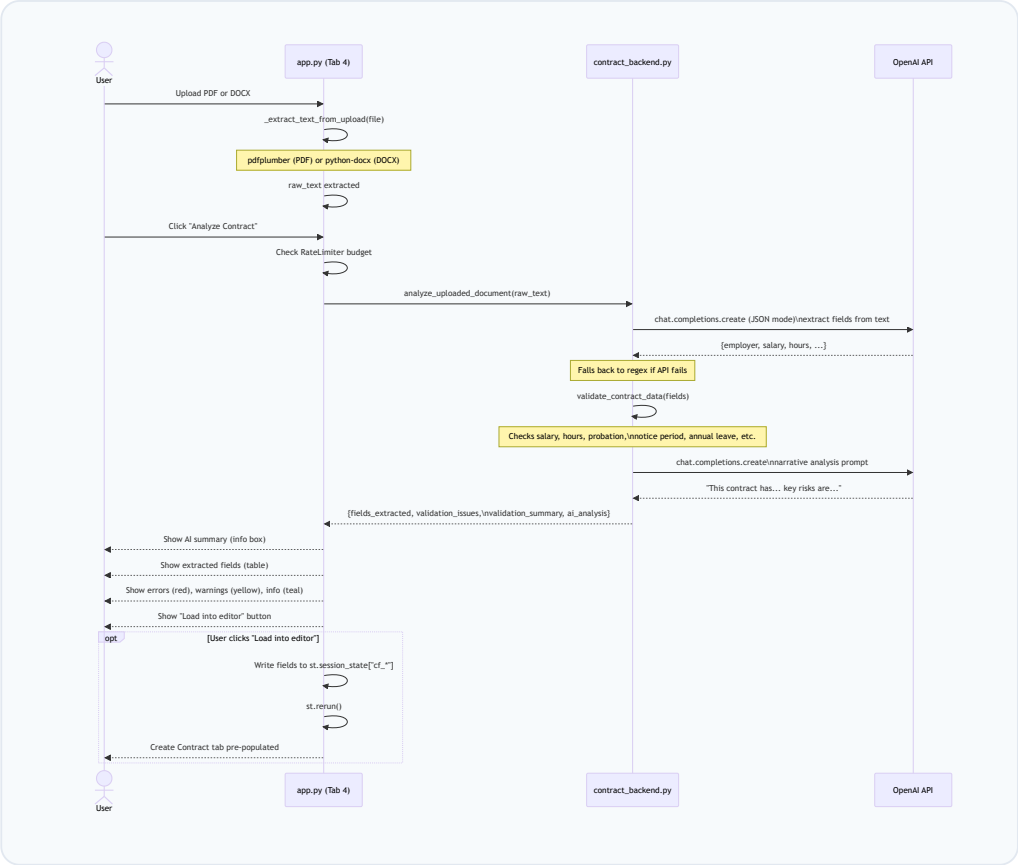
9.1 Use Case Diagram



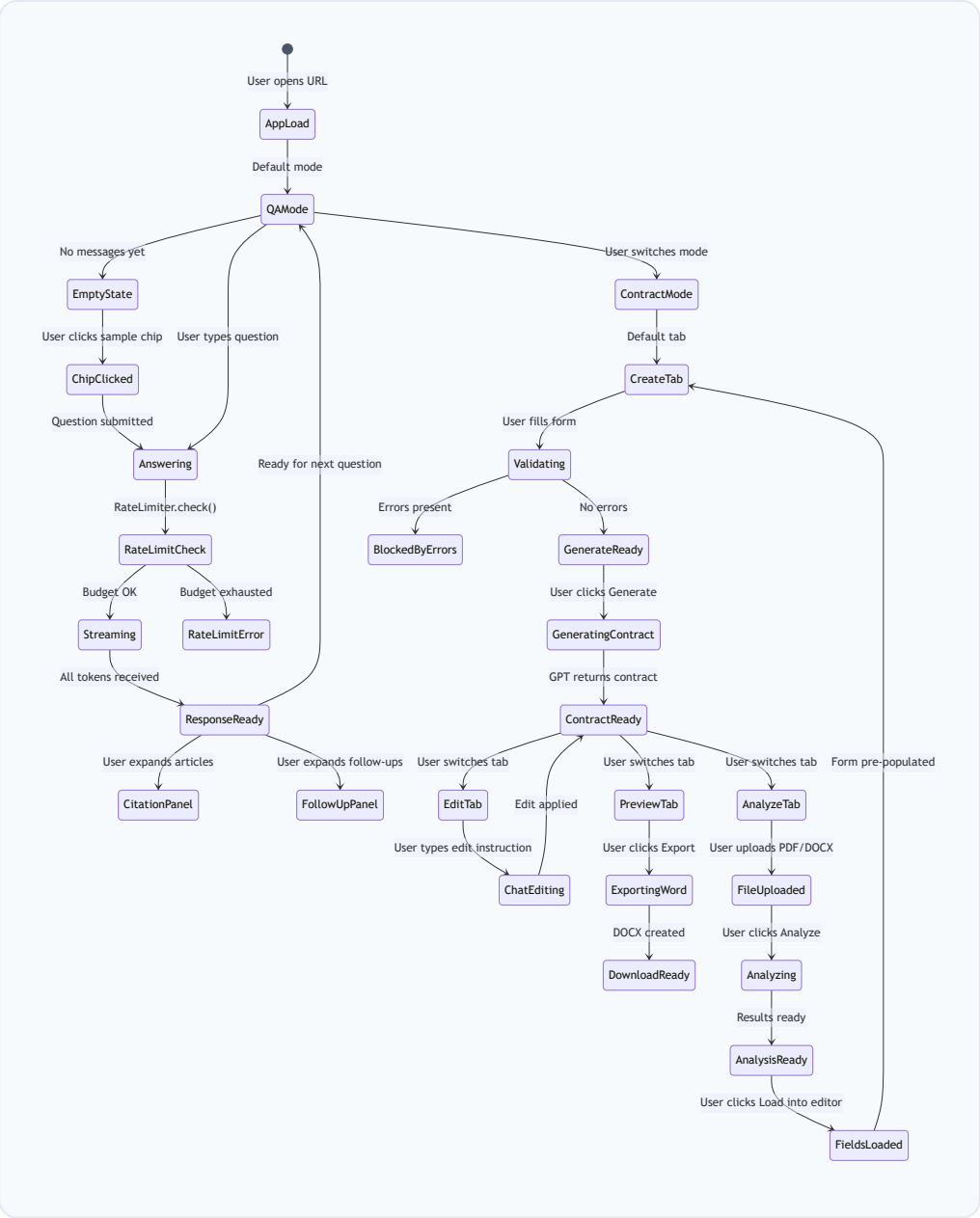
9.2 Sequence Diagram — Legal Q&A Flow



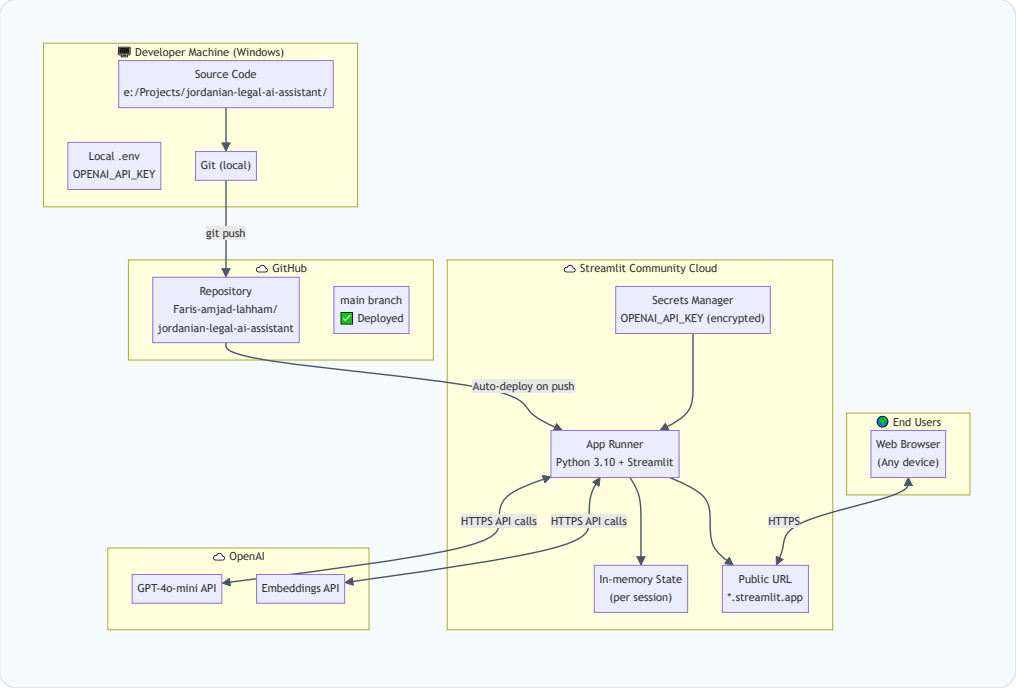
9.3 Sequence Diagram — Contract Upload & Analysis



9.4 State Machine — Application UI



9.5 Deployment Architecture



---

10. Constraints & Assumptions

10.1 Constraints

#	Constraint	Impact
C-01	GPT-4o-mini context window: 128K tokens	Limits article context passed per query
C-02	Streamlit Community Cloud: no persistent disk storage	Embedding cache rebuilt on cold start
C-03	OpenAI rate limits: 3,000 RPM (tier 1)	Concurrent users may experience delays
C-04	`langdetect` accuracy varies for short queries	Language detection may mismatch for 1–2 word queries
C-05	fpdf2 Arabic rendering: limited glyph support	Arabic PDF export may fall back to placeholder text
C-06	pdfplumber: text-only extraction	Scanned PDFs (images) cannot be analyzed

10.2 Assumptions

- Users have a modern browser with JavaScript enabled
- The OpenAI API key has sufficient quota for the expected usage volume
- Jordanian Labour Law JSON datasets are accurate and complete as of the last update

- Users understand that the system provides informational guidance only, not legal advice
- The minimum wage constant (260 JOD) is manually updated when legislation changes

---

11. Appendix

11.1 Jordanian Labour Law Constants

Constant	Value	Law Reference
Minimum wage	260 JOD/month	Ministerial Decision 2023
Max daily hours	8 hours	Article 56
Max weekly hours	48 hours	Article 56
Ramadan max hours	6 hours/day	Article 59
Annual leave (< 5 years)	14 days	Article 61
Annual leave (≥ 5 years)	21 days	Article 61
Max probation period	90 days	Article 25
Min notice period	30 days	Article 32
Overtime rate	125%	Article 60
Holiday work rate	150%	Article 60
Maternity leave	10 weeks (paid)	Article 67
Paternity leave	3 days (paid)	Article 68
Max fixed contract term	2 years (typical)	Article 25

11.2 File Structure

```
jordanian-legal-ai-assistant/
├── .gitignore
├── Legal Assistant/
│   ├── .streamlit/
│   │   ├── config.toml          # Theme + server settings
│   │   └── secrets.toml.example # Template (not real secrets)
│   ├── data/
│   │   ├── labour_law_articles_multilingual_cleaned.json
│   │   ├── labour_law_chunks_multilingual_cleaned.json
│   │   ├── labour_law_articles_ar_cleaned.json
│   │   ├── embeddings_en.npy    # Auto-generated, gitignored
│   │   └── embeddings_en_ids.json # Auto-generated, gitignored
│   ├── exports/                 # Generated DOCX files, gitignored
│   ├── app.py                   # 🔵 Main entry point
│   ├── Legal_QA.py              # 🔵 RAG + retrieval engine
│   ├── contract_backend.py      # 🔵 Contract logic + validation
│   ├── embeddings.py            # 🆕 Embedding cache module
│   ├── utils.py                 # 🆕 Rate limiter + PDF export
│   ├── evaluate_system.py       # Evaluation suite
│   └── requirements.txt
```

11.3 Revision History

Version	Date	Author	Changes
1.0	2026-04-04	Faris Amjad Lahham	Initial prototype documentation
2.0	2026-04-05	Faris Amjad Lahham	Full upgrade: hybrid retrieval, contract analysis, UI overhaul, deployment